

基于 ESP32-C3 与 MLX90640 的热成像相机系统项目报告

成果提交版 | PDF 5000 字以内 | rdfrdream

摘要

本项目设计并实现了一套基于 ESP32-C3 与 MLX90640 的热成像相机系统。系统使用 I2C 读取 MLX90640 32 x 24 红外阵列温度数据，利用 ESP32-C3 的 WiFi 能力建立局域网 Web 服务，并通过浏览器网页完成热图显示、色温条、数据刷新与 CSV 导出。由于样机阶段采用裸传感器、面包板和杜邦线连接，真实采集链路仍存在稳定性验证问题，因此展示环节结合前期实验视频和网页端演示数据说明系统流程。

1. 背景与意义

热成像可以把不可见的红外辐射转换为可观察的温度分布，在电路板发热检测、设备状态观察、教学演示和低成本测温实验中具有直观价值。传统热像仪价格较高，而 ESP32-C3 自带 WiFi/BLE，MLX90640 具有 32 x 24 阵列输出能力，二者组合适合构建低成本、可联网、可展示的热成像实验系统。

本项目的课程意义在于把微机接口技术中的 I2C 通信、嵌入式软件、串口调试、无线网络和 Web 可视化结合起来，从而形成一个完整的数据采集与显示链路。

系统总体数据链路

从红外阵列采样到浏览器热图显示，ESP32-C3 同时承担主控、无线接入和小型 Web Server。



核心接口：GPIO4/SDA、GPIO5/SCL、3.3V、GND；网页访问：AP 模式默认 192.168.4.1。

图 1 系统总体数据链路

2. 方法描述

系统采用分层设计。硬件层由 ESP32-C3、MLX90640、上拉电阻、去耦电容和面包板组成；通信层使用 I2C 总线，其中 SDA 接 GPIO4，SCL 接 GPIO5；主控层完成 MLX90640 初始化、帧数据读取、温度计算

和缓存；网络层由 ESP32-C3 建立 WiFi AP 与 HTTP Server；显示层由浏览器网页完成热图绘制和数据导出。

模块	实现内容	对应文件
采集模块	I2C 初始化、MLX90640 参数读取、温度帧计算	src/main.c, lib/MLX90640
网络模块	WiFi AP、HTTP Server、JSON 数据接口	src/main.c
显示模块	Canvas 热图、色温条、导入数据、CSV 下载	data/web_script.js
演示模块	生成平滑移动热源，用于无硬件演示	tools/mock_thermal_server.py

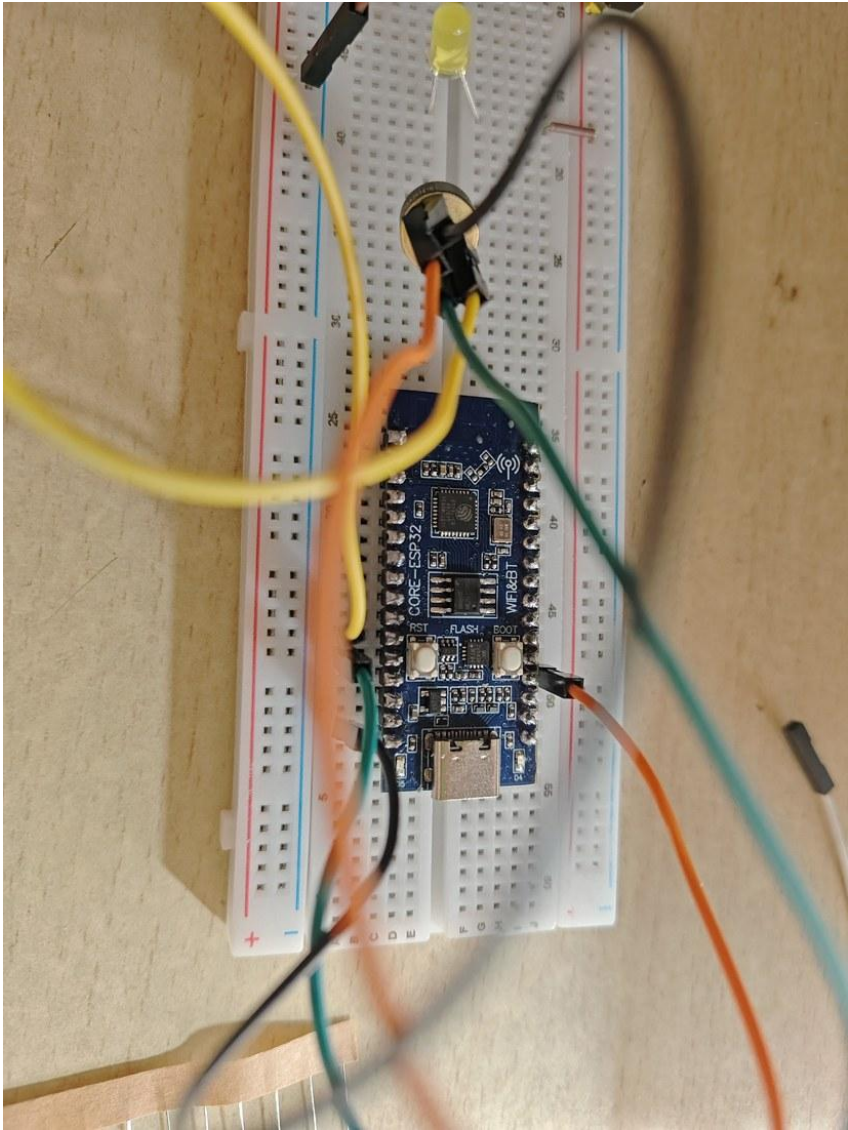


图 2 面包板样机与接线过程

3. 关键代码与截图

关键代码主要集中在三部分：第一，MLX90640 初始化与参数提取；第二，循环读取帧数据并计算温度矩阵；第三，将温度数组封装为 JSON 供网页端请求。网页端使用 JavaScript 定时获取数据，并将温度值映射为热图颜色。

```
src > C main.c > ...
264 // MLX90640 传感器初始化
265 static esp_err_t mlx90640_sensor_init(void) {
266     int status;
267     mlx90640_initialized = false;
268     mlx90640_init_status = -1;
269
270     status = MLX90640_DumpEE(MLX90640_I2C_ADDR, eeMLX90640);
271     mlx90640_eeprom_status = status;
272     if (status != 0) {
273         ESP_LOGE(TAG, "Failed to load EEPROM (status = %d)", status);
274         mlx90640_init_status = status;
275         return ESP_FAIL;
276     }
277     ESP_LOGI(TAG, "EEPROM loaded successfully.");
278
279     status = MLX90640_ExtractParameters(eeMLX90640, &mlx90640_params);
280     mlx90640_extract_status = status;
281     if (status != 0) {
282         ESP_LOGE(TAG, "Failed to extract parameters (status = %d)", status);
283         mlx90640_init_status = status;
284         return ESP_FAIL;
285     }
286     ESP_LOGI(TAG, "Parameters extracted successfully.");
287
288     // 提高刷新率到8Hz (0x04) 或 16Hz (0x05)
289     status = MLX90640_SetRefreshRate(MLX90640_I2C_ADDR, 0x05); // 16Hz
290     mlx90640_refresh_status = status;
291     if (status != 0) {
292         ESP_LOGE(TAG, "Failed to set refresh rate (status = %d)", status);
293         // 继续，即使刷新率设置失败
294     } else {
295         ESP_LOGI(TAG, "Refresh rate set to 16Hz.");
296     }
297
298     // 设置为棋盘模式 (Chess mode)
299     status = MLX90640_SetChessMode(MLX90640_I2C_ADDR);
```

图 3 MLX90640 初始化与参数读取代码

```

535 // HTTP GET处理函数 - 热成像数据
536 static esp_err_t thermal_data_get_handler(httpd_req_t *req)
537 {
538     if (!mlx90640_initialized) {
539         ESP_LOGE(TAG, "MLX90640 not initialized yet.");
540         httpd_resp_send_500(req);
541         return ESP_FAIL;
542     }
543
544     // 确保帧捕获任务正在运行
545     if (!taskRunning) {
546         start_frame_capture_task();
547     }
548
549     // 分配JSON缓冲区 - 使用静态缓冲区以避免频繁内存分配
550     static char json_buffer[JSON_BUFFER_SIZE];
551
552     // 从最新的温度数据生成JSON
553     if (xSemaphoreTake(frameMutex, pdMS_TO_TICKS(100)) == pdTRUE) {
554         int current_pos = 0;
555
556         // 使用更快的方式构建JSON
557         json_buffer[current_pos++] = '[';
558
559         for(int i = 0; i < THERMAL_DATA_SIZE; i++) {
560             // 使用1位小数来减少数据大小，同时保持足够的精度
561             int written = snprintf(json_buffer + current_pos, JSON_BUFFER_SIZE - current_pos,
562                                   "%s%.1f", i == 0 ? "" : ",", mlx90640_temperatures[i]);
563
564             if (written < 0 || written >= JSON_BUFFER_SIZE - current_pos) {
565                 ESP_LOGE(TAG, "JSON buffer overflow");
566                 xSemaphoreGive(frameMutex);
567                 httpd_resp_send_500(req);
568                 return ESP_FAIL;
569             }
570             current_pos += written;

```

图 4 热图数据接口代码：返回 JSON 温度数组

4. 实验结果

软件链路已经完成并可运行：PlatformIO 工程能够编译，ESP32-C3 固件和 SPIFFS 网页资源能够烧录，浏览器可进入热成像网页，网页端能够显示热图、色温条，并支持导入演示数据与导出 CSV。网页调试数据采用缓慢移动的热源模型，使画面变化更接近真实采样过程，便于录制演示视频。

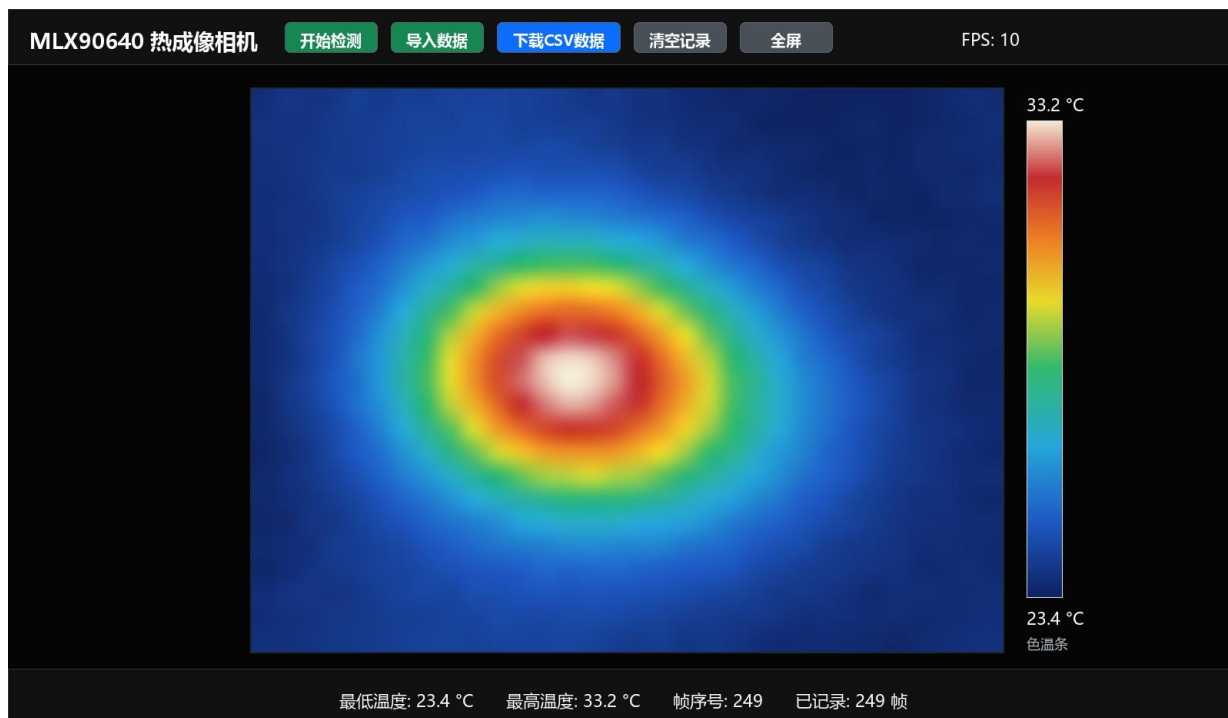


图 5 网页热图、色温条与 CSV 功能演示效果

实地验证说明

老师要求现场实地验证时，项目已完成实物搭建和网页链路验证，但 MLX90640 裸传感器与面包板连接存在松动和 I2C 稳定性问题，导致真实采集数据不够连续。因此展示时采用前期实验视频和网页端演示数据补充说明核心功能。该问题主要集中在硬件连接稳定性，而不是网页显示或软件接口逻辑。

5. 讨论与改进方向

本项目证明了 ESP32-C3 可以同时承担主控、无线通信和小型 Web Server 三个角色，比 C51 更适合联网显示，比普通 STM32 方案更少依赖外接 WiFi 模块。现阶段主要不足在于裸传感器接线和供电抗干扰能力不足，导致实时采集稳定性仍需验证。

- 硬件改进：使用 MLX90640 转接板、焊接固定或 PCB，增加外壳与传感器固定结构。
- 电气改进：优化 SDA/SCL 上拉电阻、供电去耦和线长，降低 I2C 通信错误。
- 算法改进：加入异常点滤波、帧间平滑、温度校准和热点追踪。
- 功能拓展：温度极值触发 LED 或报警器，加入阈值报警、远程访问和云端记录。
- 应用拓展：结合红外识别算法，发展为电路板发热检测或自动异常识别装置。

6. 提交材料说明

本项目已整理完整源代码、README 运行说明、项目报告、答辩 PPT 和演示材料。GitHub 仓库中 README.md 给出了编译、烧录、SPIFFS 上传、真实硬件访问、本地演示和调试接口说明。项目报告用于成果提交，答辩 PPT 用于课堂展示。

- 完整源代码：src、data、lib、tools、platformio.ini、partitions.csv。
- 项目报告：deliverables/基于 ESP32-C3 与 MLX90640 的热成像相机系统项目报告_提交版.pdf。
- 答辩 PPT：deliverables/基于 ESP32-C3 与 MLX90640 的热成像相机系统设计_答辩 PPT.pptx。
- 仓库地址：<https://github.com/1909899270h-spec/esp32c3-mlx90640-thermal-camera>。